DS4221 Final Project Proposal - Brian Hockett & Vinith Jayamani & Aidan Mayhue

Corpus

We plan on using the Carnegie Mellon University Movie Summary corpus. This dataset contains 42,306 movie plot summaries taken from Wikipedia in 2012. This dataset also contains important metadata for these movies, which includes title, release date, runtime, languages, and genres. This works well for a RAG pipeline due to the large scope of raw text, which varies enough to test the retrieval abilities of the pipeline. Three sample documents (summaries) can be found in the following gists: Hunger Games, The Lemon Drop Kid, End Game.

Questions System Will Answer

The system will be able to answer questions that relate to the general plot of movies contained in the corpus. One such question could be, "In Jaws, how do they kill the shark?" It will also be able to identify a movie based on the description of the plot, for example, "What is the space movie where the empire fights against the rebels?" Lastly, it will be able to surface movies relating to a specific topic for questions like, "What are some movies about time travel?"

Vector Index and Embedding Model

We intend to use Facebook AI Similarity Search's (FAISS) IndexIVFFlat as our vector index. Our choice of vector index is driven by the size of our corpus (~42K documents). At this scale, it is important that our chosen vector index is time efficient. As such, we prefer FAISS over microvector due to its faster run-time, and over pgvector, because our questions do not require filtering on tabular metadata. For our embedding model we will use EmbeddingGemma 2, as it is free to use, and the token limit is large enough to fit full plot summaries.

Evaluation Sketch

To evaluate our RAG pipeline we will author 25+ evaluation questions, to which we know the correct source summaries, and report the retrieval quality in terms of recall@k and mean reciprocal rank (MRR). We will compare the performance of the pipeline at differing chunk sizes, and through brute-force vs. approximate nearest neighbor search (IndexIVFFlat). Lastly, we will evaluate the quality of answers using an LLM-judge (with spot-checking).

Table Schema

Table	**Columns**	**Keys**
documents	doc_id, source, title, ingested_at, summary	PK: doc_id
chunks	chunk_id, doc_id, ordinal, text, n_tokens	PK: chunk_id FK: doc_id → documents.doc_id
embeddings	chunk_id, model, dim, vector	PK: chunk_id (model not part of PK, b.c. every embedding uses the same model) FK: chunk_id → chunks.chunk_id
eval_questions	q_id, question, relevant_doc_ids, author	PK: q_id FK: relevant_doc_ids → documents.doc_id (list → value, so not a real FK)